

*Analyse et conception orienté
objet
Unified Modeling Language
(UML)*

Pr. Sara DIOUANI

Objectifs

Situation : Un développeur ou novice en informatique souhaitant préparer son projet informatique en utilisant UML.

Besoin => Proposer une version visuelle et compréhensible du projet.

✓ Adoption du langage UML

Etapes:

- Réalisation des premières analyses de besoin du projet.
- Réalisation des premiers diagrammes définissant les éléments de base du projet : le contexte, les utilisateurs, les actions et leur déroulement.
- UML : un langage visuel qui permet d'illustrer un projet et d'échanger / communiquer sur le projet entre le développeur et le client par exemple.

Plan:

- **La programmation procédurale vs la programmation orientée objet**
- **Introduction des concepts de base de la POO**
- **Maîtriser l'analyse/conception orienté objet et le langage UML**
- **UML : Analyse et conception POO**
 - **Présentation d'UML**
 - **Introduction**
 - **Historique**
 - **Modélisation et modèle**
 - **Comment modéliser avec UML ?**
 - **Les différents types de diagrammes**
- **Comment réaliser les diagrammes**
- **L'approche objet**
- **Résumé**

La programmation procédurale vs la programmation orientée objet

- Chaque langage de programmation appartient à une “famille” de langages définissant une approche ou une méthodologie générale de programmation.
- La programmation structurée, développée dans les années 60, représentée par des langages tels Pascal, a été conçue dans le but d’organiser les programmes complexes en associant les traitements aux structures de données qu’ils manipulent.
- Ainsi, une application (ou programme) est constitué d’une partie définissant les structures de données (types, variables), d’une partie définissant les opérations (procédures, fonctions) et enfin d’un programme principal qui appelle les opérations.

- Naturellement, une procédure ou fonction peut également être à son tour un programme avec ses propres types, variables, fonctions,... Cette approche découpe une tâche en sous-programmes et permet ainsi d'analyser les problèmes de manière descendante.
- Elle met surtout *l'accent sur le traitement des programmes*, considérant les données comme des individus de deuxième ordre, tout juste bons à servir d'aliment aux procédures.
- Par exemple, le langage C est un langage de programmation procédurale car il suppose que le programmeur s'intéresse en priorité aux *traitements* que son programme devra effectuer.
- Un programmeur C commencera par identifier ces traitements pour écrire les *fonctions* qui les réalisent sur des *données* prises comme paramètres d'entrée.

- La programmation orientée-objet propose une vue différente : elle propose une méthodologie centrée sur les *données*.
- Le programmeur (exp. Java) va d'abord identifier un *ensemble d'objets*, tel que chaque objet représente un élément qui doit être utilisé ou manipulé par le programme, sous la forme d'ensembles de données.
- Ce n'est que dans un deuxième temps, que le programmeur va écrire les *traitements*, en associant chaque *traitement* à un *objet donné*.
- Un objet peut être vu comme une entité regroupant un ensemble de données et de méthodes de traitement (l'équivalent d'une fonction en C).

- **En résumé:**

Différence fondamentale entre la programmation procédurale et la programmation objet :

1- En programmation procédurale: décomposition de tâches en sous-tâches.

2- En programmation objet: on identifie les acteurs (les entités comportementales) du problème puis on détermine la façon dont ils doivent interagir pour que le problème soit résolu.

- **En résumé:**

- ✓ La programmation orientée objet, souvent abrégée POO, permet de concevoir une application sous la forme d'un ensemble de briques logicielles appelées des *objets*.
- ✓ Chaque objet joue un rôle précis et peut *communiquer avec les autres objets*.
- ✓ Les *interactions* entre les différents objets vont permettre à l'application de réaliser les *fonctionnalités* attendues.
- ✓ La POO facilite la conception de programmes par *réutilisation* de composants existants. Elle constitue le standard actuel (on parle de *paradigme*) en matière de développement de logiciels.

Introduction des concepts de base de la POO

- Nous avons l'habitude de décrire le monde qui nous entoure à l'aide de ce trio que les informaticiens se plaisent à nommer : *<entité, attribut, valeur>*
- Exemple : *<voiture, couleur, rouge>*
<TV, marque, LG>
<Ali, âge, 20>
- Ce sont les entités et non pas leurs attributs qui vous sautent aux yeux, c'est bien parce que chacune de ces entités ou objets, la voiture, la TV et la personne, se caractérise par plusieurs de ces attributs, couleur, marque, âge, prenant une valeur particulière, uniforme « sur tout l'objet ».

- L'*objet* est perçu, car il est au croisement de ces différents attributs.
- Il naît à partir de leur rencontre.
- Les *attributs* en tant que tels ne sont pas des objets, puisqu'ils servent justement à la *caractérisation* de ces objets, à les faire exister et à les rendre significatifs.

- La nature des attributs est telle qu'ils se retrouvent attribut d'un nombre important d'objets, pourtant très différents.
- Exemple:
 - Une voiture peut avoir une taille <grande> comme celle d'une personne.
 - L'arbre peut avoir une couleur <verte> comme celle d'une voiture.
- Le monde des *attributs* est beaucoup moins diversifié que le monde des *objets*.
 - => C'est une des raisons qui nous permettent de regrouper les objets en *Classes* et les classes en différentes *sous-classes*, comme nous le découvrirons plus tard.

Maitriser l'analyse/conception orienté objet et le langage UML

- Comme n'importe quel type de projet, un projet informatique nécessite une phase d'analyse, suivi d'une phase de conception.
- Dans **la phase d'analyse**, on cherche d'abord à bien comprendre et à décrire de façon précise les besoins des utilisateurs ou des clients.
 - Que souhaitent-ils faire avec le logiciel ?
 - Quelles fonctionnalités veulent-ils ?
 - Pour quel usage ?
 - Comment l'action devrait-elle fonctionner ?

C'est ce qu'on appelle « **l'analyse des besoins** ». Après validation de notre compréhension du besoin, nous imaginons la solution. C'est la partie **analyse de la solution**.
- Dans **la phase de conception**, on apporte plus de détails à la solution et on cherche à clarifier des aspects techniques, tels que l'installation des différentes parties logicielles à installer sur du matériel.

- Pour réaliser ces deux phases dans un projet informatique, nous utilisons des méthodes, des conventions et des notations.
- UML fait partie des notations les plus utilisées aujourd'hui.
- Les points clés à savoir:
 - ✓ Définir le langage UML.
 - ✓ Définir ses outils : les diagrammes.
 - ✓ Savoir comment ce langage peut contribuer à la phase d'analyse des besoins et du domaine d'un projet informatique.

UML : Analyse et conception POO

Présentation d'UML

- UML (Unified Modeling Language) est un **langage unifié pour la modélisation objet**.
- UML propose une notation et une sémantique associée à cette notation (i.e. des modèles), mais pas de processus (i.e. de démarche proposant un enchaînement d'étapes et d'activités qui mènent à la résolution d'un problème posé)
=> UML n'est pas une méthode.
- UML a une approche entièrement objet = couvrant tout le cycle de développement : analyse, conception et réalisation, et non fonctionnelle : le système est décomposé en objets collaborant (plutôt qu'en tâches décomposées en fonctions plus simples à réaliser).
- UML est surtout utilisé lorsqu'on prévoit de développer des applications avec une démarche objet (développement en Java, en C++, etc.).
- UML est utilisé pour décrire de futures applications, sans pour autant déjà être fixé sur le type de développement.

Introduction

- UML est conçu pour modéliser divers types de systèmes, de taille quelconque et pour tous les domaines d'application (gestion, temps réel, système embarqué...).
- UML permet de diviser le système d'information (d'une organisation) en le système métier et le système informatique.
- Le système métier modélise les aspects statiques et dynamiques de l'activité selon une vision externe et une vision interne (en ignorant l'implémentation technique) tandis que le système informatique recouvre la partie automatisée du système métier concrétisant les choix effectués parmi les différentes technologies d'actualité. Les concepts manipulés sont les mêmes, à chacun de ces deux niveaux d'abstraction.
- UML se compose d'une part des éléments de modélisation qui représentent toutes les propriétés du langage et d'autre part des diagrammes, qui en constituent l'expression visuelle et graphique :
 - Diagramme de cas d'utilisation,
 - Diagramme de classes,
 - Diagramme d'objets,
 - Diagramme d'états-transitions,
 - Diagramme d'activités,
 - Diagramme de séquence,
 - Diagramme de collaboration,
 - Diagramme de composants,
 - Diagramme de déploiement.

- UML n'impose pas de processus de développement logiciel particulier, même si celui sous-jacent est un processus :
 - Itératif (précisant à chaque itération les degrés d'abstraction),
 - Incrémental (i.e. en divisant le développement en étapes aboutissant chacune à la construction de tout ou partie du système),
 - Centré sur l'architecture (au niveau de la modélisation comme de la production),
 - Conduit par les cas d'utilisation (modélisant l'application à partir des modes d'utilisation attendus par les utilisateurs),
 - Piloté par les risques (afin d'écarter les causes majeures d'échec).

UML prend en compte de manière complètement intégrée l'ingénierie des besoins (cas d'utilisation).

UML est automatisable pour générer du code à partir des modèles vers les langages et les environnements de programmation.

UML est générique, extensible (en plus de couvrir les possibilités des différentes technologies objet existantes) et configurable.

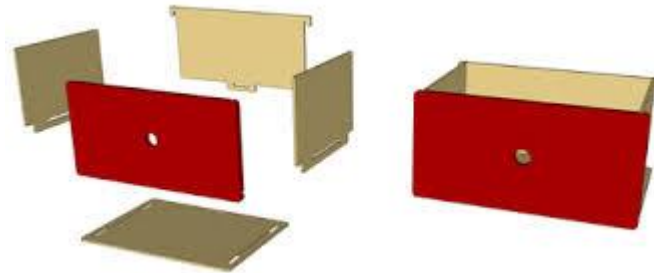
UML se veut intuitif, simple et cohérent.

Historique

- UML est né en octobre 1994 chez Rational Software Corporation à l'initiative de G. Booch et de J. Rumbaugh.
- UML 1.1 a été standardisé par l'OMG (Object Management Group) le 17 novembre 1997 suite à la demande émanant de la collaboration de plusieurs entreprises (Hewlett-Packard, IBM, i-Logix, ICON Computing, IntelliCorp, MCI Systemhouse, Microsoft, ObjecTime, Oracle, Platinum Technology, Ptech, Rational Software Corporation, Reich Technologies, Softeam, Sterling Software, Taskon et Unisys).
- La dernière version de la spécification validée par l'OMG est UML 2.5.1 (2017).
- UML est né de la fusion des trois méthodes qui ont influencé la modélisation objet au milieu des années 90 : OMT (Object Modeling Technique) , Booch et OOSE (Object-Oriented Software Engineering). Il s'agit d'un compromis qui a été trouvé par une équipe d'experts : Grady Booch, James Rumbaugh et Ivar Jacobson (surnommées les « trois amigos »).
- UML reprend en particulier les notions de partitions en sous-systèmes de Booch'93, de classes et d'associations d'OMT-2, et d'expression des besoins par les interactions entre les utilisateurs et le système d'OOSE.
- UML est à présent un standard défini par l'Object Management Group (OMG). De très nombreuses entreprises de renom ont adopté UML et participent encore aujourd'hui à son développement.

Modélisation et modèle

- Modéliser, c'est décrire de manière visuelle et graphique les besoins et les solutions fonctionnelles et techniques de votre projet logiciel. Mais modéliser pour quoi faire ?
- Avez-vous déjà eu à constituer un meuble en kit ou à brancher un nouvel équipement électronique? Vous serez d'accord que c'est plus facile à partir de schéma plutôt qu'une page de texte, non ?



Cet exemple nous démontre que l'utilisation de schémas et d'illustrations rend quelque chose de complexe plus compréhensible.

- Intéressant. Mais quel est le lien avec UML alors ?

=> C'est exactement ce à quoi UML sert dans des projets de réalisation de logiciels !

Comment modéliser avec UML ?

- Un document de texte décrivant de façon précise ce qui doit être réalisé contiendrait plusieurs dizaines de pages. En général, peu de personnes ont envie de lire ce genre de document. De plus, un long texte de plusieurs pages est source d'interprétations et d'incompréhension.

=> UML nous aide à faire cette **description de façon graphique** et devient alors un excellent moyen pour « **visualiser** » le(s) futur(s) logiciel(s).

- Cas :
- Supposons que vous rencontrez un informaticien qui commencent à programmer sans avoir étudié le besoin des utilisateurs, du client, etc. et sans avoir fait une bonne et adéquate conception de l'application.
- Quand vous faites la remarque, vous recevez des réponses du type :
- On n'est pas vraiment obligé de modéliser un logiciel que l'on doit réaliser!
- À quoi bon modéliser le logiciel, puisque je sais exactement ce qu'il faut !
- C'est une perte de temps.
- La réalisation de cette application est urgente.
- Etc.

- La meilleure réponse est de réfléchir à l'analogie suivante:
- Est-ce que c'est possible de laisser un maître d'œuvre commencer la construction d'une maison sans avoir préalablement réaliser des plans par un architecte ?

La réponse est bien sûr que non, à moins que l'on veuille bien accepter une maison qui ne réponde pas à nos besoins, qui soit déformée ou qui le deviendrait dans le future, et dans laquelle il serait difficile de faire des travaux faute de plans.

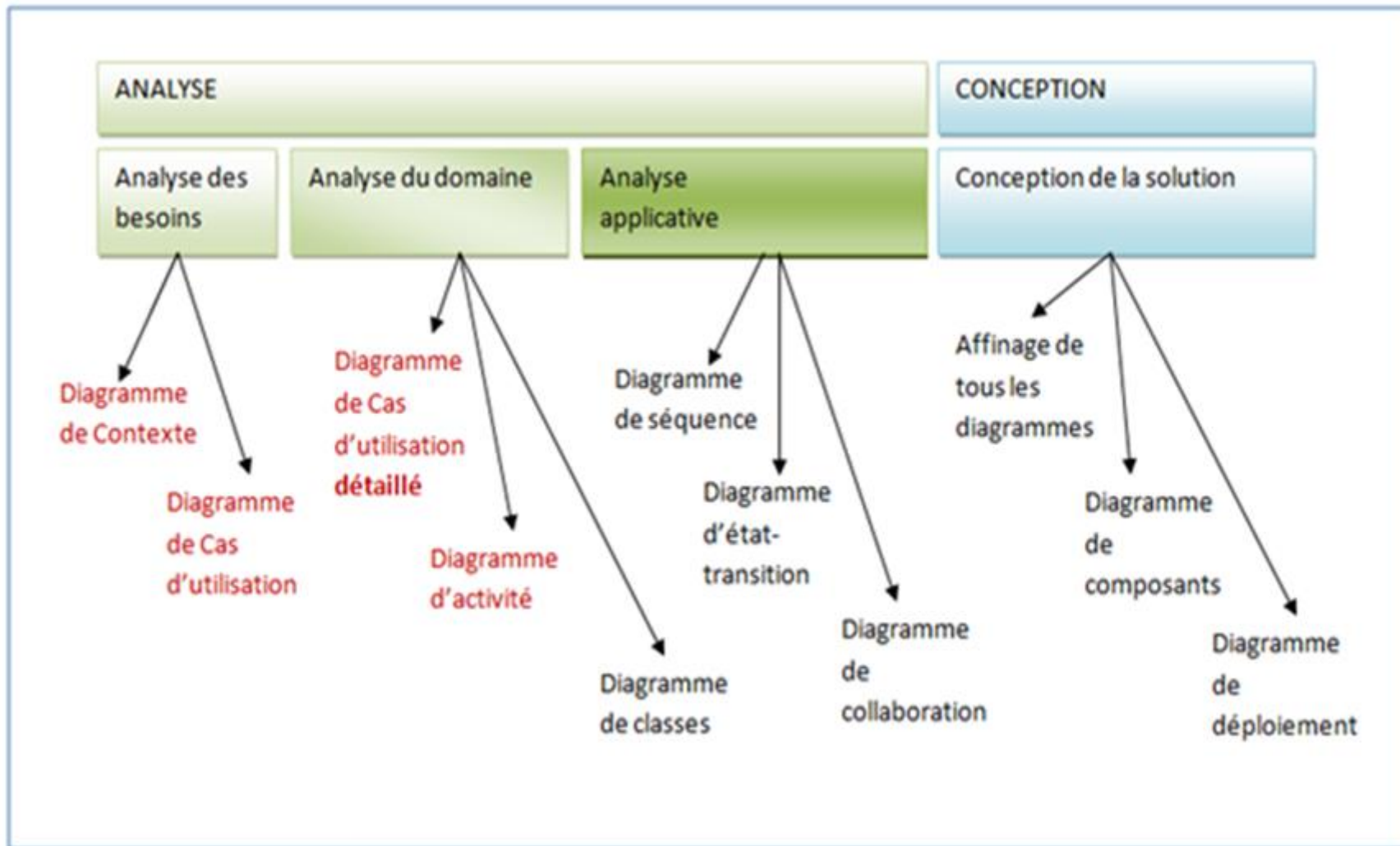
- Une solution informatique qui a été réalisée sans analyse et sans conception (étapes où l'on modélise la future application) risque elle aussi de ne pas répondre aux besoins, de comporter des anomalies et d'être très difficile à maintenir.

=> Tout comme la construction d'une maison nécessite des plans à différents niveaux (vision extérieure, plan des différents étages, plans techniques...), la réalisation d'un logiciel ou d'un ensemble de logiciels a besoin d'un certain nombre de **diagrammes**.

- Le temps que prend la modélisation ne devrait pas être une limite. Cette étape nous permet de ne pas perdre davantage de temps ultérieurement :
- ✓ Pour « faire coller » un logiciel déjà développé aux besoins (qu'on n'avait pas étudié, ni compris) ;
- ✓ pour comprendre comment un logiciel a été créé avant d'y apporter des modifications.

Les différents types de diagrammes

- UML, c'est l'acronyme anglais de « Unified Modeling Language ». On le traduit par « Langage de modélisation unifié ».
- La notation UML est un **langage visuel** constitué d'un ensemble de schémas, appelés des **diagrammes**, qui donnent chacun une vision différente du projet à traiter.
- UML nous fournit donc des diagrammes pour **représenter** le logiciel à développer : son fonctionnement, sa mise en route, les actions susceptibles d'être effectuées par le logiciel, etc.
- Réaliser ces diagrammes revient donc à **modéliser les besoins** du logiciel à développer.



Positionnement des diagrammes dans la démarche de projet

- Il existe **13 diagrammes** « officiels ». Ces diagrammes sont tous réalisés **à partir du besoin des utilisateurs** et peuvent être regroupés selon les deux aspects suivants :
- **Les aspects fonctionnels** : Qui utilisera le logiciel et pour quoi faire ? Comment les actions devront-elles se dérouler ? Quelles informations seront utilisées pour cela ?
- **Les aspects liés à l'architecture** : Quels seront les différents composants logiciels à utiliser (base de données, librairies, interfaces, etc.) ? Sur quel matériel chacun des composants sera installé ?

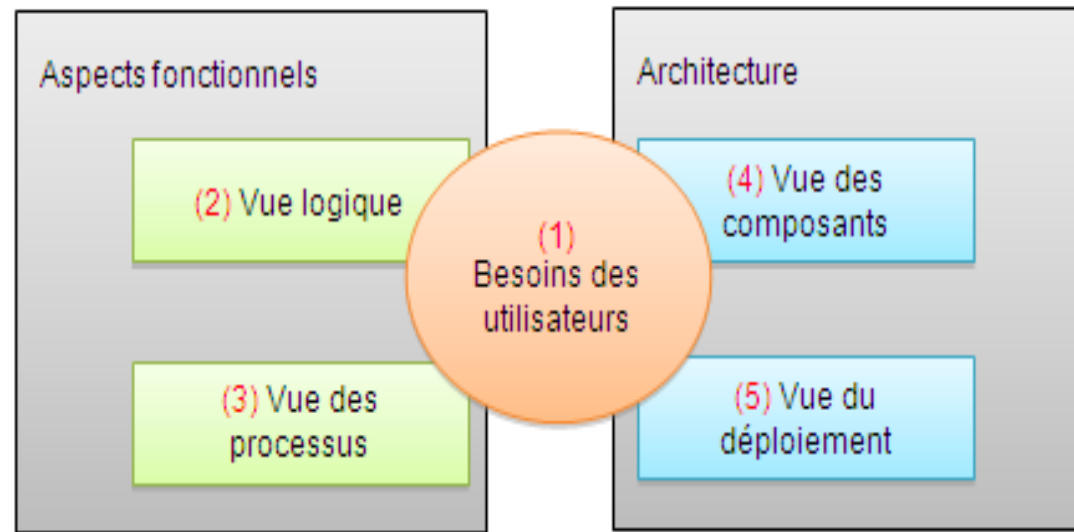
- UML modélise donc le système logiciel suivant ces deux modes de représentation.
- Nous allons donc dans un premier temps décrire ces différents aspects d'un logiciel grâce au **schéma 4+1 vues** et parcourir brièvement les différents diagrammes qui les composent.

Les 4+1 vues d'un système

La conception d'un logiciel est organisée autour des aspects fonctionnels et d'architecture. Ces deux aspects sont représentés par le schéma de 4 vues, axées sur les besoins des utilisateurs (parfois intitulé des cas d'utilisation), appelé **4+1 vues**.

Une première décomposition d'une problématique ou système peut donc être faite à l'aide de 4+1 vues. Le schéma ci-dessous montre les différentes vues permettant de répondre au mieux aux besoins des utilisateurs, organisées selon les deux aspects (fonctionnels et architecture). Chacune des vues est constituée de diagrammes.

- Nous nous concentrerons uniquement sur les besoins des utilisateurs, au centre de la figure.



Les 4+1 vues d'un système

Comment réaliser les diagrammes?

- Vous avez le choix ! Vous pouvez soit reprendre les normes de ces diagrammes que nous verrons au fur et à mesure de ce cours et les dessiner à la main, soit utiliser des logiciels gratuits ou payants pour les réaliser.
- Exemple: StarUml, ArgoUml, BoUml, PowerDesigner, etc.
- Le langage UML ne préconise aucune démarche, ce n'est donc pas une méthode. Chacun est libre d'utiliser les types de diagramme qu'il souhaite, dans l'ordre qu'il veut. Il suffit que les diagrammes réalisés soient cohérents entre eux, avant de passer à la réalisation du logiciel.

L'approche objet

- Dans la gestion de projet, nous pouvons citer deux approches permettant de définir les besoins :
- La décomposition fonctionnelle (ou l'approche procédurale)
- L'approche objet (sur laquelle est basée UML)

La décomposition fonctionnelle

Avant l'apparition de l'approche objet dans les années 80, une autre démarche était largement utilisée.

Pendant longtemps, de nombreux logiciels étaient conçus par l'approche fonctionnelle descendante, appelée également la décomposition fonctionnelle.

L'approche par décomposition fonctionnelle considère que le logiciel est composé d'une **hiérarchie de fonctions et de données**. Les fonctions fournissent les services désirés et les données représentent les informations manipulées.

- Exemple : logiciel Word

- Une fonction ou fonctionnalité est une option mise à disposition des utilisateurs. Lorsque vous utilisez le Word, vous disposez de plusieurs fonctionnalités qui sont regroupées dans des menus et des onglets. Par exemple, dans l'onglet « Accueil », vous trouverez une fonctionnalité qui permet de modifier la couleur du texte, une autre pour rechercher un mot dans le texte, etc.

- Pour réaliser une fonction du logiciel, on peut utiliser un ensemble d'autres fonctions, déjà disponibles, à condition qu'on rende ces dernières suffisamment génériques.

- Le Word contient la fonction « Enregistrer sous », qui utilise très probablement la fonction « Enregistrer ». Cette fonction se sert de l'emplacement du nom du fichier connu à l'ouverture.

- Pour la fonction « Enregistrer sous », l'utilisateur doit préciser un endroit où enregistrer tout en donnant éventuellement un autre nom au fichier. L'enregistrement se fait alors exactement de la même manière que dans la fonction « Enregistrer ».

- Ce découpage n'a pas que des avantages. **Les fonctions sont alors interdépendantes** : une simple mise à jour du logiciel à un point donné, peut impacter **en cascade** d'autres fonctions.
- On peut éviter cela en faisant attention à créer des fonctions très génériques. Mais respecter cette contrainte rend l'écriture du logiciel et sa maintenance plus complexe.
- Par exemple, si les développeurs de la société Microsoft décident de modifier le déroulement de la fonction « Enregistrer », cela impactera forcément la fonction « Enregistrer sous ». On peut aisément imaginer qu'une modification dans une fonction qui est utilisée par un grand nombre d'autres fonctions aura un grand impact.

Ayant constaté les problèmes inhérents à la décomposition fonctionnelle, de nombreux experts se sont penchés sur une approche différente.

=>De cette réflexion est née **l'approche objet**, dont UML s'inspire largement.

- L'approche objet est une démarche qui s'organise autour de 4 principes fondamentaux. C'est une démarche :
 - ✓ itérative et incrémentale ;
 - ✓ guidée par les besoins du client et des utilisateurs ;
 - ✓ centrée sur l'architecture du logiciel ;
 - ✓ qui décrit les actions et les informations dans une seule entité.

Reprenons notre exemple de la construction d'une maison. L'architecte réalise toujours plusieurs versions des plans **avant d'avoir la version définitive** (c'est-à-dire celui accepté par le client).

Après un premier entretien avec son client, il réalise les plans pour la vision extérieure de la maison et les plans de surface. Ces plans sont soumis au client et donnent généralement lieu à des discussions.

Cela permet à l'architecte de mieux cerner le besoin de son client. Il apporte alors les modifications qui s'imposent aux plans qu'il avait réalisés. Il s'agit là d'une **première itération** sur les plans de la vision extérieure et les plans de surface.

Par la suite, l'architecte réalisera les plans techniques qui seront nécessaires pour les différents corps de métier (les maçons, les électriciens, les plombiers, etc.). Il arrive parfois que ces plans mettent en évidence certaines contraintes qui obligent l'architecte à revenir sur les plans de surface (par exemple pour ajouter l'emplacement d'une colonne d'aération qui n'avait pas été prévue au départ). Il y a donc une **deuxième itération** sur les plans de surface.

Une démarche **itérative** c'est donc faire des allers-retours entre le plan initial et les modifications apportées par les acteurs du projet.

De la même façon, la modélisation d'un logiciel se fera en plusieurs fois. Les diagrammes ne seront pas réalisés de façon complètement satisfaisante dès la première fois.

⇒ Il nous faudra plusieurs versions d'un même diagramme pour obtenir la version finale. Chaque version est le fruit d'une itération (un nouveau passage sur les diagrammes). Une itération permet donc d'affiner la compréhension du futur logiciel.

- La démarche est **guidée par les besoins** du client et des utilisateurs.

Les besoins d'un client, dans un projet de logiciel sont les exigences exprimées par le client au niveau des fonctionnalités, du rendu et des actions d'un logiciel.

Par exemple, lors de la réalisation du Word, les clients ont probablement souhaité un logiciel qui leur permettrait d'écrire du texte, de l'effacer, d'intégrer des images, d'enregistrer du contenu, le modifier, le supprimer, etc.

Les besoins des utilisateurs servent de fil rouge, tout au long du cycle de développement :

- lors de la phase d'analyse pour la clarification, l'affinage et la validation des besoins des utilisateurs ;
- lors de la phase de conception et de réalisation pour la vérification de la prise en compte des besoins des utilisateurs ;
- lors de la phase de test afin de garantir la satisfaction des besoins.

La démarche est également **centrée sur l'architecture du logiciel**, élément fondamental du développement.

L'architecture logicielle décrit les choix stratégiques qui déterminent en grande partie les qualités du logiciel (adaptabilité, performances, fiabilité...).

=> L'approche objet **décrit aussi bien les actions que les informations dans une même entité.**

- Les différents diagrammes utilisés en UML donnent tous une vision particulière du logiciel à développer. Parmi ces diagrammes, il en est un qui représente particulièrement bien ce qui est à développer si on opte pour un développement objet. Il s'agit du **diagramme de classes**.

Retenons que:

- Le diagramme de classes donnera une vision assez claire des informations qui seront utilisées par le logiciel, mais également des fonctions (ou opérations) qui devront s'appuyer sur ces informations.
- L'approche objet mélange donc l'analyse des informations et des actions au lieu de les analyser séparément.
- Le diagramme de classes sera étudié en détail, plus tard avec les autres diagrammes d'UML.

Résumé

- UML signifie « Unified Modeling Language » ou Langage de modélisation unifié en français. C'est un langage de modélisation qui permet de représenter graphiquement les besoins des utilisateurs. On utilise pour cela des diagrammes.
- UML est une démarche qui se base sur une approche objet. Cette approche s'appuie sur 4 principes fondamentaux.
- L'approche objet nécessite une démarche itérative et incrémentale, c'est-à-dire que le concepteur doit faire des allers-retours entre les diagrammes initiaux et, les besoins du client et des utilisateurs perçus au fur et à mesure de la conception du logiciel afin de le modifier si nécessaire.
- L'approche objet est guidée par les besoins du client.
- L'approche objet est centrée sur le diagramme de classes qui décrit aussi bien des actions que des informations dans une même entité. Les autres diagrammes nous aident à voir clair dans les besoins et dans la solution qui est à développer. Ils permettent de compléter le diagramme de classes.

*Analyse et conception orienté
objet
Unified Modeling Language
(UML)*

Pr. Sara DIOUANI